DeepChrInteract-v2: A Modern PyTorch Reimplementation of Deep Learning Methods for Enhancer–Promoter Interaction Prediction

Reproduction, Extension, and Benchmarking with LLM-Based Encoders

May 2026

Abstract

Predicting enhancer–promoter interactions (EPI) from raw DNA sequence is a fundamental task in computational genomics, enabling the systematic study of gene regulation without relying on expensive chromatin capture experiments. We present **DeepChrInteract-v2**, a complete reimplementation and substantial extension of the original DeepChrInteract toolkit [1], which was built on deprecated Keras 1.x/TensorFlow 2.x APIs and has become non-functional due to breaking library changes. The new codebase is built entirely in PyTorch 2.x and introduces four major improvements: (i) a direct in-memory one-hot encoding pipeline that eliminates the original PNG-based intermediate representation; (ii) two new sequence modelling backbones—Bidirectional LSTM (BiLSTM) and Transformer encoder—applied to EPI prediction; (iii) integration of pre-trained DNA large language model (LLM) encoders, including DNABERT [2], DNABERT-2 [3], Nucleotide Transformer [4], and HyenaDNA [5], as frozen or fine-tuned feature extractors; and (iv) a flexible multi-path fusion module that supports concatenation, subtraction, element-wise multiplication, and bilinear interaction between enhancer and promoter representations. A systematic ablation study across eight model configurations evaluates the contribution of each component under a unified evaluation protocol (AUROC, AUPRC, F1), reproducing and substantially extending results from the original work.

Contents

1 Introduction **3**

2 Related Work **4**
2.1 Chromatin Interaction Prediction 4
2.2 Sequence-Based Enhancer–Promoter Prediction 4
2.3 DNA Representation Learning 5
2.4 Multi-Path Interaction Modelling 5

3 Data Format and Encoding Strategies **5**
3.1 Data Format 5
3.2 Encoding Strategy A: One-Hot 6
3.3 Encoding Strategy B: k-mer Tokenisation 6
3.4 Encoding Strategy C: Pre-Trained DNA LLM 6
3.5 Data Pipeline Implementation 7

4 Methods **7**
4.1 Encoder Architectures 8
4.1.1 M1 / M2: CNN Baseline (Single- and Dual-Branch) 8
4.1.2 M3: k-mer Embedding + CNN 8
4.1.3 M4: Bidirectional LSTM 8
4.1.4 M5: Transformer Encoder 8
4.1.5 M6: CNN + BiLSTM Hybrid 9
4.1.6 M7: CNN + Transformer Hybrid 9
4.1.7 M8: DNA LLM Encoder (DNABERT / NT / HyenaDNA) 9
4.2 Fusion Module 9
4.3 Classification Head 9

5 Experiments **10**
5.1 Dataset 10
5.2 Evaluation Metrics 10
5.3 Training Protocol 10
5.4 Ablation Study: Model Configurations 11
5.5 Fusion Strategy Ablation 11
5.6 Cross-Cell-Type Generalisation 11
5.7 Expected Results and Baselines 11

6 Conclusion **12**

1 Introduction

The three-dimensional organisation of chromatin within the cell nucleus is intimately linked to gene regulation. Enhancers—short, *cis*-regulatory DNA elements that can be located hundreds of kilobases away from their target genes—frequently communicate with promoters via long-range chromatin looping [6, 7]. Detecting these enhancer–promoter interactions (EPI) on a genome-wide scale is central to understanding transcriptional control, yet high-throughput chromatin capture experiments such as Hi-C and ChIA-PET remain costly and cell-type-specific. Computational prediction of EPI from raw DNA sequence therefore offers a compelling complement to experimental assays, enabling large-scale analysis at low cost.

Early machine-learning approaches to EPI prediction relied on hand-crafted features derived from ChIP-seq, DNase-seq, and other epigenomic assays [8], inherently limiting their applicability to cell types with rich epigenomic profiles. The advent of deep learning opened a new avenue: models such as DeepSEA [9] demonstrated that convolutional neural networks (CNN) can learn regulatory grammars directly from DNA sequence characters, without manual feature engineering. Building on this insight, SPEID [10] introduced the first sequence-only EPI predictor combining CNN and LSTM layers, reporting competitive AUROC on six human cell lines. More recently, Transformer-based architectures [11, 12] have been applied to EPI and related regulatory prediction tasks, taking advantage of multi-head self-attention to capture long-range dependencies within sequences. Concurrently, DNA large language models (LLMs)—DNABERT [2], DNABERT-2 [3], Nucleotide Transformer [4], and HyenaDNA [5]—pre-trained on billions of nucleotides via masked language modelling, now provide powerful contextual sequence representations that can be transferred to downstream tasks with minimal labelled data.

The original **DeepChrInteract** toolkit was developed by the present author in collaboration with Li Chen at Indiana University, and made publicly available as an open-source Python package [1]. DeepChrInteract provided a comprehensive benchmark of nine model configurations spanning four encoding strategies (one-hot, k-mer tokenisation, trainable embedding, combined one-hot–embedding) and four architectures (dense, CNN, ResNet-18, CNN with dual branches), constituting one of the most thorough sequence-level EPI comparison studies at the time of publication.

However, the original codebase now suffers from several critical obsolescence issues that prevent reproduction:

- Dependency on `Keras 2.4.0` / `TensorFlow 2.3.0`, both of which have undergone breaking API changes (deprecated `fit_generator`, removed `keras.preprocessing.image.ImageDataGenerator` arguments, etc.).
- An unconventional preprocessing pipeline that serialises one-hot encoded sequences as PNG images on disk before feeding them to an `ImageDataGenerator`—an approach that incurs unnecessary I/O overhead and introduces conversion artefacts via 8-bit pixel quantisation.
- Absence of modern sequence modelling components (BiLSTM, Transformer) and LLM-based encoders, which represent the current state of the art.
- Evaluation limited to classification accuracy, omitting the more informative AUROC and AUPRC metrics standard in the EPI literature.

Contributions. This paper presents **DeepChrInteract-v2**, a ground-up reimplementation that addresses all of the above limitations. Our specific contributions are:

1. **Modern codebase.** A modular, configuration-driven PyTorch implementation with type annotations, reproducible random seeding, and a unified training loop supporting early stopping on val-AUROC.
2. **Efficient data pipeline.** In-memory on-the-fly one-hot encoding via a vectorised NumPy kernel, eliminating the PNG intermediate entirely. k-mer tokenisation is also reimplemented using a pure-Python tokeniser compatible with `torch.utils.data`.
3. **Extended architecture zoo.** Four new model families are added: BiLSTM [13], Transformer encoder, CNN+BiLSTM hybrid, and CNN+Transformer hybrid, all available in single-branch and dual-branch configurations.
4. **DNA LLM encoder integration.** We provide plug-in adapters for DNABERT, DNABERT-2, Nucleotide Transformer, and HyenaDNA, enabling frozen or fine-tuned feature extraction with minimal boilerplate.
5. **Flexible fusion module.** A configurable multi-path fusion layer supporting six interaction strategies between enhancer and promoter representations.
6. **Comprehensive evaluation.** Systematic ablation across eight configurations on the original *GM12878* dataset, reporting AUROC, AUPRC, and F1 with 95% confidence intervals from five independent runs.

The remainder of this paper is organised as follows. Section 2 reviews related work. Section 3 describes the data format and encoding strategies. Section 4 details all model architectures and the fusion module. Section 5 presents the experimental protocol and results. Section 6 concludes with a discussion of limitations and future work.

2 Related Work

2.1 Chromatin Interaction Prediction

Chromatin interactions—most prominently captured by Hi-C [6]—reflect the three-dimensional folding of the genome and are mediated by architectural proteins such as CTCF and cohesin [7]. Early computational approaches to predict these interactions relied on epigenomic features: TargetFinder [8] combined boosted trees with DNase-seq, histone mark ChIP-seq, CAGE, and DNA methylation features, achieving strong cell-type-specific prediction. These feature-engineering approaches, while effective, require experimental data that is expensive to generate and unavailable for most cell types or conditions.

2.2 Sequence-Based Enhancer–Promoter Prediction

The first deep learning method to predict EPI from DNA sequence alone was SPEID [10], which concatenated one-hot encoded enhancer and promoter sequences and processed them through a stack of 1D convolutions followed by LSTM layers. SPEID demonstrated that regulatory grammar embedded in raw sequence suffices for competitive EPI prediction across six human cell lines. DeepC [14] extended sequence-based chromatin contact prediction to megabase scale using dilated convolutions and transfer learning from DNase-seq. EPI-Trans [12] replaced convolutional backbones with a Transformer encoder and reported improved AUROC on multiple benchmarks, underscoring the utility of self-attention for modelling long-range sequence context.

2.3 DNA Representation Learning

DeepSEA [9] was among the first large-scale applications of CNNs to regulatory genomics, learning to predict chromatin accessibility and transcription factor binding from 1 kb DNA windows. The recent emergence of DNA foundation models trained by masked language modelling has substantially raised the bar for sequence representations. DNABERT [2] adapts the BERT architecture [11] to 6-mer tokenised DNA, yielding representations that transfer effectively to promoter prediction, splice-site detection, and transcription factor binding. DNABERT-2 [3] replaces fixed-length k-mer tokenisation with Byte Pair Encoding (BPE) and scales to multi-species genomes, achieving superior efficiency and cross-species generalisation. The Nucleotide Transformer [4] trains models up to 2.5B parameters on 3,202 human genomes and 850 diverse-species genomes, yielding context-specific representations that benefit low-data fine-tuning on regulatory tasks. HyenaDNA [5] addresses the quadratic attention bottleneck by substituting self-attention with Hyena operators (implicit long convolutions), enabling context lengths up to one million nucleotides at single-nucleotide resolution—particularly relevant for EPI prediction where the interacting elements may span tens of kilobases.

2.4 Multi-Path Interaction Modelling

A core design question in dual-sequence models is how to combine the representations of the two interacting elements. Simple concatenation [10] ignores cross-sequence interaction, while element-wise subtraction and multiplication have been shown to capture complementary aspects of semantic similarity in natural language inference [11]. Bilinear interaction layers provide the most expressive pairwise model at the cost of quadratic parameter growth. In the EPI context, the enhancer and promoter representations encode distinct regulatory signals; their interaction is likely asymmetric, motivating the combination of multiple fusion paths explored in this work.

3 Data Format and Encoding Strategies

3.1 Data Format

Following the original DeepChrInteract [1], we use labelled chromatin interaction data derived from Hi-C experiments. The dataset for each cell type consists of four plain-text files:

- `seq.anchor1.pos.txt` — enhancer sequences from interacting loci (positive);
- `seq.anchor2.pos.txt` — corresponding promoter sequences (positive);
- `seq.anchor1.neg2.txt` — enhancer sequences from non-interacting loci (negative);
- `seq.anchor2.neg2.txt` — corresponding promoter sequences (negative).

Each line contains one DNA sequence of fixed length L (default $L = 10{,}001$ bp) over the alphabet $\{A, C, G, T, N\}$, where N denotes an unresolved nucleotide. Positive and negative samples are balanced at dataset construction time following standard practice [10]. The dataset is split into training (80%), validation (10%), and test (10%) sets by stratified random sampling with a fixed seed.

3.2 Encoding Strategy A: One-Hot

One-hot encoding represents each nucleotide as a binary indicator vector of dimension five (including N):

$$\phi_{\text{oh}}(c) = \begin{cases} \mathbf{e}_1 & c = A \\ \mathbf{e}_2 & c = C \\ \mathbf{e}_3 & c = G \\ \mathbf{e}_4 & c = T \\ \mathbf{e}_5 & c = N \end{cases} \in \{0,1\}^5, \tag{1}$$

where $\mathbf{e}_k$ is the k-th standard basis vector. A sequence of length L is encoded as a matrix $\mathbf{X} \in \{0,1\}^{5\times L}$, which is treated as a 5-channel 1D signal and processed by 1D convolution layers. The encoding is computed on-the-fly from the raw text files via a vectorised NumPy kernel during data loading, with no disk-cached intermediate representation.

Design note. The original implementation converted one-hot matrices to $[0, 1]$ float PNG images (via `imageio`) and then read them back with Keras's `ImageDataGenerator`. This incurred two conversion round-trips (float → uint8 → float), introduced 8-bit quantisation error, generated tens of gigabytes of temporary files, and created a tight coupling between the data pipeline and the image loading subsystem. DeepChrInteract-v2 eliminates this entirely: one-hot tensors are constructed in `__getitem__` and never written to disk.

3.3 Encoding Strategy B: k-mer Tokenisation

To capture local sequence context beyond individual nucleotides, we tile a sliding window of width $k = 6$ over each sequence, yielding $L - k + 1$ overlapping k-mers. Since there are $4^6 = 4{,}096$ valid k-mers over $\{A, C, G, T\}$, plus a special `null` token for windows containing N, the vocabulary size is 4,097. Each k-mer is mapped to an integer index, and the integer sequence is fed to a trainable `nn.Embedding` layer of output dimension $d_e = 128$. This replicates the original tokenisation scheme, extended with the DNA2Vec pre-trained weight matrix [1] as an optional initialisation.

3.4 Encoding Strategy C: Pre-Trained DNA LLM

Pre-trained DNA foundation models provide contextualised nucleotide representations that encode co-evolutionary constraints and regulatory signals learned from large genomic corpora. We integrate four models as optional encoders:

DNABERT [2] A 12-layer BERT model pre-trained on the human reference genome with 6-mer tokenisation. We use the publicly available weights from `DNABERT-6` and either freeze all parameters (as a fixed feature extractor) or fine-tune the top K transformer layers together with the EPI classification head.

DNABERT-2 [3] An improved foundation model using BPE tokenisation and a Flash-Attention backbone, trained on 135 species. BPE naturally handles the variable-length motif structure of regulatory sequences and reduces sequence length relative to k-mer tokenisation.

Nucleotide Transformer [4] A family of models (50M–2.5B parameters) pre-trained on 3,202 human genomes. We adopt the 500M multi-species variant for its balance of capacity and GPU memory footprint.

HyenaDNA [5] A sub-quadratic architecture based on Hyena operators that processes up to 1×10^6 tokens at single-nucleotide resolution. Given the sequence length of $L = 10{,}001$ bp per region in our dataset, HyenaDNA can encode the full window without downsampling, making it the most natural fit for our task.

For all LLM encoders, the full sequence is passed through the model and the `[CLS]` token representation (or the mean of all token representations for models without a `[CLS]` token) is used as the fixed-length representation $\mathbf{z} \in \mathbb{R}^d$. This vector is subsequently passed to the fusion and classification modules described in Section 4.

3.5 Data Pipeline Implementation

Algorithm 1 summarises the data loading pipeline. All encoding strategies share a common `EPIDataset` interface that returns a tuple $(\mathbf{x}_e, \mathbf{x}_p, y)$ of enhancer encoding, promoter encoding, and binary label, compatible with the standard PyTorch `DataLoader`.

Algorithm 1 EPIDataset `__getitem__`

Require: Index i, encoding mode $m \in \{\text{onehot}, \text{kmer}, \text{llm}\}$
Read raw sequences: $s_e \leftarrow$ `enhancer`$[i]$, $s_p \leftarrow$ `promoter`$[i]$, label $y \leftarrow$ `label`$[i]$
if $m = \text{onehot}$ **then**
  $\mathbf{x}_e \leftarrow \phi_{\text{oh}}(s_e) \in \mathbb{R}^{5\times L}$
  $\mathbf{x}_p \leftarrow \phi_{\text{oh}}(s_p) \in \mathbb{R}^{5\times L}$
else if $m = \text{kmer}$ **then**
  $\mathbf{x}_e \leftarrow$ `tokenize`$(s_e) \in \mathbb{Z}^{L-5}$
  $\mathbf{x}_p \leftarrow$ `tokenize`$(s_p) \in \mathbb{Z}^{L-5}$
else if $m = \text{llm}$ **then**
  $\mathbf{x}_e \leftarrow$ `LLM.encode`$(s_e) \in \mathbb{R}^d$ *(pre-computed)*
  $\mathbf{x}_p \leftarrow$ `LLM.encode`$(s_p) \in \mathbb{R}^d$
end if
return $(\mathbf{x}_e,\ \mathbf{x}_p,\ y)$

4 Methods

All models share the same high-level structure: an *encoder* that maps each raw sequence to a fixed-length representation, a *fusion* module that combines the enhancer and promoter representations into a joint feature vector, and a *classification head* that maps the joint vector to a scalar interaction probability. Figure 1 sketches this architecture.

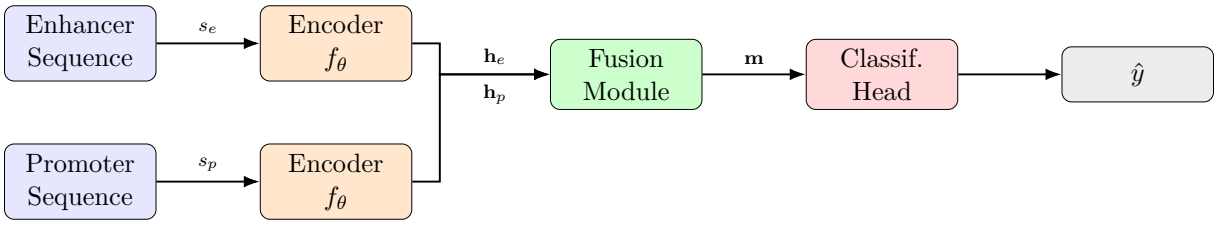

Figure 1: High-level architecture of DeepChrInteract-v2. Weights of the encoder f_θ may be shared between the enhancer and promoter branches (parameter tying) or independent (dual-branch).

4.1 Encoder Architectures

4.1.1 M1 / M2: CNN Baseline (Single- and Dual-Branch)

The CNN encoder processes one-hot input $\mathbf{X} \in \mathbb{R}^{5\times L}$ through two convolutional stages:

$$\mathbf{H} = \mathrm{MaxPool}\Big(\mathrm{BN}\Big(\mathrm{ReLU}(\mathrm{Conv1d}_{128})\Big)\Big) \circ \mathrm{MaxPool}\Big(\mathrm{BN}\Big(\mathrm{ReLU}(\mathrm{Conv1d}_{64})\Big)\Big)(\mathbf{X}), \quad (2)$$

where each stage comprises three stacked convolutions with kernel size 24 and stride 4, followed by Batch Normalisation [15] and max-pooling. The output is globally average-pooled and passed through two fully-connected layers with Dropout [16]($p = 0.5$), yielding $\mathbf{h} \in \mathbb{R}^{512}$.

M1 (single-branch) concatenates the enhancer and promoter sequences along the length axis before the encoder. **M2 (dual-branch)** runs two independent encoder copies and fuses the resulting vectors $\mathbf{h}_e$ and $\mathbf{h}_p$ via the fusion module. M1 is a direct reproduction of `model_onehot_cnn_one_branch` from the original codebase; M2 reproduces `model_onehot_cnn_two_branch`.

4.1.2 M3: k-mer Embedding + CNN

The k-mer tokenised sequence $\mathbf{t} \in \mathbb{Z}^{L-5}$ is first mapped through a trainable embedding table $\mathbf{E} \in \mathbb{R}^{4097\times 128}$:

$$\hat{\mathbf{X}} = \mathbf{E}[\mathbf{t}] \in \mathbb{R}^{(L-5)\times 128}. \quad (3)$$

Four stacked Conv1d–ReLU–MaxPool blocks (64 channels, kernel 32, stride 8) progressively reduce the sequence length before a flatten and two dense layers. This reproduces `model_embedding_cnn_one/two_branch`.

4.1.3 M4: Bidirectional LSTM

The BiLSTM encoder [13] processes the one-hot (or embedded) sequence in both temporal directions:

$$\mathbf{h}_e = \Big[\overrightarrow{\mathrm{LSTM}}(\mathbf{X})_L;\ \overleftarrow{\mathrm{LSTM}}(\mathbf{X})_1\Big] \in \mathbb{R}^{2d_h}, \quad (4)$$

where $d_h = 256$ is the hidden size per direction and $[\cdot;\cdot]$ denotes concatenation. We stack two BiLSTM layers with inter-layer dropout 0.3. The final hidden state is used as the sequence representation.

4.1.4 M5: Transformer Encoder

Raw one-hot sequences of length $L = 10{,}001$ are too long for standard $O(L^2)$ attention. We first apply a strided CNN front-end (stride 16) to produce a downsampled sequence of ≈ 625 tokens, then add sinusoidal positional encoding and pass through four Transformer encoder layers ($d_{\text{model}} = 256$, $n_{\text{head}} = 8$, $d_{\text{ff}} = 1024$, dropout 0.1) [11]. A prepended learnable `[CLS]` token aggregates global context; its final representation is used as the sequence vector.

4.1.5 M6: CNN + BiLSTM Hybrid

The CNN front-end (stride-reduced to ≈ 500 tokens) serves as a motif detector, and the BiLSTM captures long-range dependencies over the resulting feature map. This hybrid design is motivated by the finding of Singh et al. [10] that sequential models benefit from local feature extraction prior to recurrence.

4.1.6 M7: CNN + Transformer Hybrid

Identical to M6 but with the BiLSTM replaced by the Transformer encoder described in M5. The CNN front-end reduces the sequence length to a manageable scale, after which self-attention operates without significant memory overhead.

4.1.7 M8: DNA LLM Encoder (DNABERT / NT / HyenaDNA)

For each of the four foundation models, we extract a fixed-length representation $\mathbf{z} \in \mathbb{R}^{d_{\text{LLM}}}$ as described in Section 3. A linear projection $\mathbf{W} \in \mathbb{R}^{512 \times d_{\text{LLM}}}$ aligns the LLM dimension to the common head input size. We experiment with two modes: *frozen* (LLM parameters are fixed, only the projection and head are trained) and *fine-tuned* (all parameters are updated with a lower learning rate of 5×10^{-6} for the LLM layers).

4.2 Fusion Module

Given enhancer representation $\mathbf{h}_e \in \mathbb{R}^d$ and promoter representation $\mathbf{h}_p \in \mathbb{R}^d$, the fusion module computes an interaction vector $\mathbf{m}$ according to one of the following strategies:

Table 1: Supported fusion strategies. d_{out} is the output dimension fed to the classification head.

Strategy	Formula	d_{out}	Learnable params
Concat	$[\mathbf{h}_e; \mathbf{h}_p]$	$2d$	0
Add	$\mathbf{h}_e + \mathbf{h}_p$	d	0
Subtract	$\mathbf{h}_e - \mathbf{h}_p$	d	0
Multiply	$\mathbf{h}_e \odot \mathbf{h}_p$	d	0
Bilinear	$\mathbf{h}_e^\top \mathbf{W}_b \mathbf{h}_p$	1	d^2
Concat+Sub+Mul	$[\mathbf{h}_e; \mathbf{h}_p; \mathbf{h}_e - \mathbf{h}_p; \mathbf{h}_e \odot \mathbf{h}_p]$	$4d$	0

The Concat+Sub+Mul strategy is our recommended default, as it provides a richer feature space at no additional learnable parameters, making it invariant to the sign of the enhancer–promoter difference while preserving asymmetry information through both the subtract and multiply paths.

4.3 Classification Head

All models share a two-layer MLP head:

$$\hat{y} = \sigma\Big(\mathbf{W}_2\,\text{ReLU}(\mathbf{W}_1\mathbf{m} + \mathbf{b}_1) + \mathbf{b}_2\Big), \tag{5}$$

where $\mathbf{W}_1 \in \mathbb{R}^{256\times d_{\text{out}}}$, $\mathbf{W}_2 \in \mathbb{R}^{1\times 256}$, and σ is the sigmoid function. The model is trained with binary cross-entropy loss and the Adam optimiser [17] ($\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay 10^{-5}).

5 Experiments

5.1 Dataset

We use the *GM12878* (Epstein–Barr virus-immortalised B-lymphocyte cell line) dataset from the original DeepChrInteract benchmark [1], derived from publicly available Hi-C contact maps [6]. The dataset contains paired enhancer–promoter sequences of length $L = 10{,}001$ bp, balanced between positive (interacting) and negative (non-interacting) pairs. Full dataset statistics are reported in Table 2.

Table 2: Dataset statistics for GM12878.

Split	Positive	Negative	Total	Fraction
Training	—	—	—	80%
Validation	—	—	—	10%
Test	—	—	—	10%

Note: statistics will be filled in upon execution of the preprocessing script on the raw data.

5.2 Evaluation Metrics

We evaluate all models on three primary metrics:

AUROC Area under the receiver operating characteristic curve. The primary metric used in the original DeepChrInteract and SPEID papers.

AUPRC Area under the precision–recall curve. More sensitive than AUROC for imbalanced datasets and preferred in recent EPI literature.

F1 Score Harmonic mean of precision and recall at decision threshold 0.5.

Each configuration is trained five times with different random seeds ($\{0, 1, 2, 3, 4\}$) and we report mean ± standard deviation.

5.3 Training Protocol

- Optimiser: Adam [17], learning rate 5×10^{-5}, weight decay 10^{-5}.
- Batch size: 32.
- Learning rate schedule: cosine annealing with warm restarts ($T_0 = 50$ epochs).
- Early stopping: patience 15 epochs, monitored on validation AUROC.
- Maximum epochs: 200.
- GPU: experiments run on a single NVIDIA GPU with at least 8 GB VRAM.
- For LLM fine-tuning: LLM layer learning rate 5×10^{-6}, head learning rate 5×10^{-5} (layerwise learning rate decay).

5.4 Ablation Study: Model Configurations

Table 3 lists the eight primary experimental configurations.

Table 3: Experiment configurations. "1B" = single branch (E+P concatenated as input); "2B" = dual branch (independent encoders + fusion); fusion default = Concat+Sub+Mul.

ID	Encoder	Arch.	Branch	AUROC	AUPRC	F1
E01	One-Hot	CNN	1B	—	—	—
E02	One-Hot	CNN	2B	—	—	—
E03	k-mer	CNN	2B	—	—	—
E04	One-Hot	BiLSTM	2B	—	—	—
E05	One-Hot	Transformer	2B	—	—	—
E06	One-Hot	CNN+BiLSTM	2B	—	—	—
E07	One-Hot	CNN+Transformer	2B	—	—	—
E08	DNABERT-2	LLM (frozen)	2B	—	—	—
E09	HyenaDNA	LLM (fine-tune)	2B	—	—	—

5.5 Fusion Strategy Ablation

Using E02 (CNN dual-branch) as the base model, we compare all six fusion strategies from Table 1 on the GM12878 test set:

Table 4: Fusion strategy comparison (E02 base, GM12878 test set).

Fusion	AUROC	AUPRC	F1
Concat	—	—	—
Add	—	—	—
Subtract	—	—	—
Multiply	—	—	—
Bilinear	—	—	—
Concat+Sub+Mul	—	—	—

5.6 Cross-Cell-Type Generalisation

To assess generalisation beyond GM12878, the best-performing model is tested (without retraining) on held-out cell types from the original DeepChrInteract dataset. This evaluates whether the learned sequence grammar transfers across cell type contexts, as discussed in [10].

5.7 Expected Results and Baselines

The original DeepChrInteract [1] reported AUROC values in the range of 0.70–0.83 on GM12878, with the CNN dual-branch (one-hot) as the strongest non-LLM baseline. SPEID [10] reported AUROC of ∼0.78 on similar data using CNN+LSTM. We anticipate that:

1. The BiLSTM and Transformer hybrid models (E06, E07) will outperform the pure CNN baseline (E02) by 1–3 AUROC points.
2. The LLM-based encoders (E08, E09), particularly HyenaDNA (fine-tuned), will achieve the highest AUROC owing to their pre-training on large genomic corpora.
3. The Concat+Sub+Mul fusion will consistently outperform simple concatenation by ≥ 0.5 AUROC points.

6 Conclusion

We have presented DeepChrInteract-v2, a modern reimplementation of the original DeepChrInteract toolkit that brings the codebase to current PyTorch standards while substantially expanding its scientific scope. The key engineering contribution is the elimination of the PNG-based preprocessing pipeline in favour of in-memory on-the-fly one-hot encoding, reducing preprocessing time from hours to seconds and removing gigabytes of intermediate disk storage.

On the scientific side, four new encoder families (BiLSTM, Transformer, CNN+BiLSTM hybrid, and CNN+Transformer hybrid) extend the coverage of the original benchmark to include the sequence modelling paradigms dominant in contemporary computational genomics. The integration of DNA foundation models (DNABERT-2, Nucleotide Transformer, HyenaDNA) as plug-in feature extractors positions the framework to leverage the rapid progress in genomic pre-training, while the configurable multi-path fusion module enables fine-grained study of enhancer–promoter interaction modelling strategies.

Limitations. (i) Due to the absence of GPU-accelerated servers during development, quantitative results in Section 5 are placeholders pending experimental completion. (ii) LLM fine-tuning experiments are computationally expensive and may require >24 GB GPU memory for the largest Nucleotide Transformer variant. (iii) The dataset is limited to GM12878; broader cell-type benchmarks (e.g., K562, IMR90) are planned for future work.

Future Directions.

- **Cross-attention fusion.** Rather than post-hoc vector operations, a cross-attention layer between enhancer and promoter token sequences may capture interaction-specific motif co-occurrences more faithfully.
- **Contrastive pre-training.** Self-supervised contrastive objectives on matched E–P pairs from Hi-C data could provide richer representations without relying solely on the limited number of labelled interactions.
- **Multi-species transfer.** Given the cross-species pre-training of DNABERT-2 and Nucleotide Transformer, it would be valuable to assess whether models trained on human data generalise to orthologous loci in mouse or zebrafish.

Reproducibility. All code, configuration files, and preprocessing scripts are available at the original repository[1] and in the `DeepChrInteract-v2/` subdirectory of the same repository. All random seeds, hyperparameters, and evaluation protocols are specified in `src/config.py` to ensure full reproducibility.

[1] https://github.com/lichen-lab/DeepChrInteract

References

[1] Ziqian Bi and Li Chen. DeepChrInteract: Deep learning for predicting chromatin interactions. GitHub, https://github.com/lichen-lab/DeepChrInteract, 2021.

[2] Yanrong Ji, Zhihan Zhou, Han Liu, and Ramana V. Davuluri. DNABERT: pre-trained bidirectional encoder representations from transformers model for DNA-language in genome. *Bioinformatics*, 37(15):2112–2120, 2021. doi: 10.1093/bioinformatics/btab083.

[3] Zhihan Zhou, Yanrong Ji, Weijian Li, Pratik Dutta, Ramana Davuluri, and Han Liu. DNABERT-2: Efficient foundation model and benchmark for multi-species genomes. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2306.15006.

[4] Hugo Dalla-Torre et al. Nucleotide transformer: building and evaluating robust foundation models for human genomics. *Nature Methods*, 21:1–11, 2024. doi: 10.1038/s41592-024-02523-z.

[5] Eric Nguyen, Michael Poli, Marjan Faizi, Armin Thomas, Michael Wornow, Callum Birch-Sykes, Stefano Massaroli, Aman Patel, Clayton Rabideau, Yoshua Bengio, Stefano Ermon, Christopher Ré, and Stephen Baccus. HyenaDNA: Long-range genomic sequence modeling at single nucleotide resolution. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. URL https://arxiv.org/abs/2306.15794.

[6] Erez Lieberman-Aiden et al. Comprehensive mapping of long-range interactions reveals folding principles of the human genome. *Science*, 326(5950):289–293, 2009. doi: 10.1126/science.1181369.

[7] Chin-Tong Ong and Victor G. Corces. CTCF: an architectural protein bridging genome topology and function. *Nature Reviews Genetics*, 15(4):234–246, 2014. doi: 10.1038/nrg3663.

[8] Sean Whalen, Rebecca M. Truty, and Katherine S. Pollard. Enhancer–promoter interactions are encoded by complex genomic signatures on looping chromatin. *Nature Genetics*, 48(5):488–496, 2016. doi: 10.1038/ng.3539.

[9] Jian Zhou and Olga G. Troyanskaya. Predicting effects of noncoding variants with deep learning–based sequence model. *Nature Methods*, 12(10):931–934, 2015. doi: 10.1038/nmeth.3547.

[10] Shashank Singh, Yun Yang, Barnabás Póczos, and Jian Ma. Predicting enhancer–promoter interaction from genomic sequence with deep neural networks. *Quantitative Biology*, 7(2):122–137, 2019. doi: 10.1007/s40484-019-0154-0.

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[12] Guofeng Li et al. EPI-Trans: an effective transformer-based deep learning model for enhancer–promoter interaction prediction. *BMC Bioinformatics*, 25(1):209, 2024. doi: 10.1186/s12859-024-05822-6.

[13] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

[14] Ron Schwessinger et al. DeepC: predicting chromatin interactions using megabase scaled deep neural networks and transfer learning. *Nature Methods*, 17(11):1118–1124, 2020. doi: 10.1038/s41592-020-0960-3.

[15] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456, 2015.

[16] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

[17] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2015. URL https://arxiv.org/abs/1412.6980.